

# R

# Mòdul 1

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Introducció al programari R</b>                      | <b>2</b>  |
| 1.1      | Presentació . . . . .                                   | 2         |
| 1.2      | Primer contacte amb R . . . . .                         | 3         |
| <b>2</b> | <b>Estadística amb R</b>                                | <b>5</b>  |
| 2.1      | Recollida, organització i resum de dades . . . . .      | 5         |
| 2.2      | Conclusions i decisions a partir de les dades . . . . . | 6         |
| <b>3</b> | <b>Entorn de treball del programari R</b>               | <b>7</b>  |
| 3.1      | Ajuda amb R . . . . .                                   | 7         |
| 3.2      | Formats de fitxer . . . . .                             | 7         |
| 3.2.1    | Manipular taules i gràfics . . . . .                    | 7         |
| 3.2.2    | Format “.Rhistory”: Històric . . . . .                  | 9         |
| 3.2.3    | Format “.Rdata”: Àrea de treball . . . . .              | 10        |
| 3.2.4    | Format “.R”: Scripts . . . . .                          | 11        |
| 3.3      | Construccions i conceptes generals . . . . .            | 12        |
| <b>4</b> | <b>Exercicis</b>                                        | <b>14</b> |

# Part 1

## Introducció al programari R

R és un programari molt útil per a la manipulació, anàlisi i representació gràfica de dades. És *open source*, gratuït, i es pot trobar a

<http://www.r-project.org>

on hi ha, a més del codi font, els executables per a una varietat de sistemes operatius. El més pràctic és baixar els executables (*binaries*) per al sistema operatiu adequat.

Donat que R és un codi obert, dia a dia es van publicant paquets que estenen la seva configuració bàsica. A mesura que apareixen noves versions, obtenim una configuració bàsica més completa i actual. Tot i així, sovint ens caldrà *instal·lar* i *carregar* funcions addicionals al R.

Suposeu que volem carregar un paquet anomenat **car**, ja que conté una funció que necessitem. Executem la següent comanda:

```
>install.packages("car")
```

Amb aquesta instrucció hem afegit a la documentació de R el paquet **car**. Ara, per tal d'utilitzar les funcions que conté, caldrà carregar-lo a la sessió de treball amb la següent comanda:

```
>library(car)
```

Ara ja disposem de les funcions i taules de dades del paquet **car**.

A la xarxa hi trobareu molt de material complementari, apunts, tutorials, etc. Com a llibre de text, recomanem el manual en format *pdf* que podeu descarregar de la pàgina oficial de R:

[cran.r-project.org/doc/contrib/R-intro-1.1.0-espanol.1.pdf](http://cran.r-project.org/doc/contrib/R-intro-1.1.0-espanol.1.pdf)

i com a tutorial, per la seva claredat d'exposició, us recomanem consultar la pàgina [www.r-tutor.com](http://www.r-tutor.com).

### 1.1 Presentació

L'any 1993 Robert Gentleman i Ross Ihaka, del Departament d'Estadística de la Universitat d'Auckland, van desenvolupar el llenguatge de programació R. Es tractava d'un projecte de software lliure, resultat de la implementació GNU del llenguatge S. Actualment, el seu desenvolupament és responsabilitat del Development Core Team.

Cal ressaltar les següents característiques de R:

- Proporciona una gran varietat d'eines estadístiques, que el converteixen en un dels programes més complets que tenim a l'abast.

- Permet a l'usuari estendre'l, tot definint les seves pròpies funcions. La majoria de les funcions de R estan escrites en el seu propi llenguatge.
- Per a algorismes computacional-exigents es poden implementar biblioteques amb C, C++, o Fortran, que es carreguen directament.
- I encara més: Programació orientada a objectes. Diversitat de bases de dades. Format de documentació: LaTeX. Potent eina de càlcul numèric.

## 1.2 Primer contacte amb R

R sap fer alguns càlculs:

```
>3+2
[1] 5
```

També sap fer càlculs més complicats, com ara  $e^{-2}$ :

```
>exp(-2)
[1] 0.1353353
```

Per a veure el resultat amb més precisió podem utilitzar la comanda `print( )`, modificant el nombre de dígitos (el màxim és 22):

```
>print(exp(-2), digits=16)
[1] 0.1353352832366127
```

Fins aquí una calculadora de butxaca ho fa igual de bé. Compliquem-ho una mica introduint *variables*, és a dir noms que poden contenir valors numèrics concrets:

```
>x <- 3
>x + 2
[1] 5
>x * (x+2)
[1] 15
```

Observeu que les *assignacions*, com ara `x <- 3`, no produeixen cap resposta. Per a saber el valor que té una variable, hem d'escriure una instrucció amb el seu nom.

Una variable no només pot contenir números; també pot contenir un *vector* de números:

```
>weight <- c(60, 72, 57, 90, 95, 72)
>weight
[1] 60 72 57 90 95 72
```

Els vectors es construeixen mitjançant la *funció* `c()`. Si voleu saber què vol dir el `[1]`, que apareix abans de cada sortida, construïu un vector llarg, de manera que al demanar el valor de la seva variable, el resultat ocupi més d'una línia.

Suposem que els valors que hem introduït a la variable **weight** són els pesos de sis persones en kg i que les seves alçades, en metres i donades en el mateix ordre, són les que posarem ara a la nova variable **height**:

```
>height <- c(1.75, 1.80, 1.65, 1.90, 1.74, 1.91)
>bmi <- weight/height^2
>bmi
[1] 19.59184 22.22222 20.93664 24.93075 31.37799 19.73630
```

O bé, si volem més precisió:

```
>print(bmi, digits=10)
[1] 19.59183673 22.22222222 20.93663912 24.93074792 31.37798917 19.73630109
```

A la segona línia estem dividint el pes pel quadrat de l'alçada, obtenint un valor conegut com a “body mass index”. Observeu que la variable `bmi`, on hem dipositat el resultat, conté també un vector. La fórmula s'ha aplicat component a component, als dos vectors involucrats. Aquesta és una característica fonamental de R: Les operacions entre vectors es realitzen component a component. Una altra característica, relacionada, és l'anomenada “regla de re-utilització”: Al fer operacions entre vectors, si un d'ells té una longitud més curta que els altres, quan arribem al final es torna a llegir des del principi. Per exemple, `height*c(1,-1)` produeix un vector on els components parells de `height` queden amb el signe canviat (proveu-ho!). Aquesta característica també l'hem utilitzat a la fórmula de la segona línia: El 2 que representa elevar al quadrat, cal considerar-lo com un vector d'un sol component, que anem re-utilitzant fins que s'acaba l'altre vector.

Al tancar una sessió de R (aturant el programa), ens demana si volem guardar l'espai de treball (*workspace*). L'espai de treball conté, en particular, el valor de les variables que tenim definides fins al moment, de manera que la propera vegada que engegum el R siguin estant disponibles. Proveu d'aturar i tornar a engegar, i comproveu que les variables que teníeu conserven el seu valor.

## Part 2

# Estadística amb R

L'Estadística és la ciència que té per objecte la *recollida, organització i resum de dades*, així com la *obtenció de conclusions* i la *presa de decisions* a partir d'elles.

L'Estadística Descriptiva és la part que s'ocupa del *resum i organització*, i no necessita cap teoria matemàtica prèvia. En canvi l'Estadística Inferencial, que pretén treure conclusions sobre una *Població* a partir d'una *Mostra* (una petita part de la població), necessita l'estudi previ del Càlcul de Probabilitats.

### 2.1 Recollida, organització i resum de dades

Les dades amb les quals treballem són el resultat de mesurar (en un sentit ample) una o vàries variables a una població, és a dir, el resultat de la *recollida de dades*. Les mesures es poden prendre de diverses maneres, depenent del *protocol de mesura* que utilitzem. Així, podem obtenir valors numèrics, alfanumèrics, etc. Amb R podem treballar amb qualsevol format de dades.

En general organitzem les mesures preses a una població, i.e. les dades, en forma de taules (o matrius), on les variables indexen les columnes i els casos (o individus) indexen les files. Exemple: Considerem certa població de la qual hem triat individus per a mesurar-los el pes i l'altura. Com organitzem el resultat de les mesures? Considerem una taula, que anomenarem **TaulaExemple**, que tindrà dues columnes: una columna l'anomenarem **pes** i l'altra **altura**. La taula tindrà tantes files com individus hàgim mesurat. Anem a fer-ho amb R:

```
>pes <- c(60, 72, 57, 90, 95, 72)
>altura <- c(1.75, 1.80, 1.65, 1.90, 1.74, 1.91)
>TaulaExemple <-data.frame(pes,altura)
```

Ara podríem afegir una altra mesura a la nostra taula, per exemple, el sexe de cadascun dels individus que representen cadascuna de les files. Anomenem **sexe** a aquesta nova variable:

```
>sexe <- c("dona","home","dona","dona","home","home")
>TaulaExemple <-data.frame(TaulaExemple,sexe)
```

Fixeu-vos que la classe de la variable **pes** és diferent de la classe de la variable **TaulaExemple** i de la variable **sexe**:

```
>class(pes)
>class(TaulaExemple)
>class(sexe)
```

Les classes dels objectes que manipulem amb R s'han de tenir presents, ja que depenent de la classe podrem accedir a unes o d'altres funcions. Si enlloc d'introduir el sexe com una mesura alfanumèrica

l'haguéssim codificat posant, per exemple, 1 per indicar home i 2 en altre cas, aleshores la classe hagués estat diferent. Fixeu-vos:

```
>sexe_cod <- c(2,1,2,2,1,1)
>class(sexe_cod)
```

En aquest cas, com que els valors numèrics indiquen factors o categories, caldrà canviar la classe de la variable:

```
>sexe_cod <- factor(sexe_cod)
>class(sexe_cod)
```

Fixeu-vos que hem codificat la variable **sexe** manualment, i seria convenient poder-ho fer automàticament. Per això tenim la funció **recode**, que es troba en el paquet **car**. Primer anem a carregar el paquet (no cal instal·lar-lo, si ja ho havíeu fet)

```
>library(car)
>recode(sexe, "'home'=1;else=2")
```

En funció del tipus de valors que prenen les nostres dades i de la mesura que representen, podem decidir quina és la millor manera de resumir-les. Hi ha dos procediments principals:

- Procediment numèric o quantitatiu
- Procediment categòric o qualitatiu

Aquests dos procediments són flexibles i en alguns casos convindrà combinar-los per tal d'obtenir una bona descripció de les dades. Ara bé, en la majoria dels casos, un dels dos serà suficient. Hi ha tres tipus de descriptors clau que permeten resumir o descriure unes dades: els estadístics, les taules de freqüències i els gràfics.

## 2.2 Conclusions i decisions a partir de les dades

L'estadística inferencial requereix l'ús de la probabilitat així com d'altres processos que s'han de fer tenint molta cura. El programari R és molt potent per a fer estadística inferencial. Ara bé, com amb qualsevol programa només disposem de les funcions programables. El treball de modelar (per tal de treure conclusions i prendre decisions) és feina de l'usuari. L'usuari haurà de tenir un coneixement teòric exhaustiu d'aquestes funcions abans d'utilitzar-les.

## Part 3

# Entorn de treball del programari R

L'entorn de treball més usual de R és la consola, és a dir, per línies de comanda. Aquest és l'entorn per defecte. Tot i així, un cop familiaritzats amb la consola es recomana conèixer l'ús d'altres entorns de treball amb R que ens facilitaran tasques específiques, els més importants:

- **RStudio**, l'entorn gràfic **que utilitzarem a les pràctiques d'aquest curs**.
- **JGR** o Java GUI for R, terminal multiplataforma basat en Java.
- **Rcmdr** o R Commander i **DeduceR**, interfícies gràfiques multiplataforma.
- **Rexcel**, que ens permet usar R i Rcmdr des de l'Excel

Per exemple, mitjançant R Commander o Deducer podem accedir a les funcions bàsiques de modelització estadística a través de menús, és a dir, sense utilitzar línies de comandes.

Abans de començar una sessió de R és imprescindible que us organitzeu i establiu un *directori de treball* per interactuar amb R, on R *llegirà i escriurà*.

### 3.1 Ajuda amb R

Aquí teniu algunes comandes que us ajudaran:

- `apropos("text")` (no oblideu les cometes), dona una llista de comandes que contenen la paraula clau `text`.
- `help(comanda)`, `help("comanda")` o `?comanda`.
- `help.start()` obre el navegador en una pàgina local d'informació sobre R.

### 3.2 Formats de fitxer

Com a formats propis de R tenim essencialment: `".R"`, `".Rdata"` i `".Rhistory"`. Per a treballar els formats de fitxers propis de R, anem a començar manipulant fitxers de dades.

#### 3.2.1 Manipular taules i gràfics

A través de la consola de R podem manipular diferents formats de fitxers de bases de dades. Els formats més usuals que utilitzarem són: `"csv"`, `"txt"`, `"dat"`. Tot i així, podríem fer tasques més específiques com importar de l'Excel fitxers de dades en format `"xls"` o bé fitxers `"sav"` de SPSS, etc.

Si volem obrir i/o guardar fitxers amb aquests formats, on hi tenim emmagatzemades taules de dades, utilitzarem essencialment les instruccions `scan`, `read` i `read.table` per a llegir-les, `edit` per a visualitzar-les i modificar-les i `write` i `write.table` per a guardar-les.

Si el que volem guardar és un gràfic o altres sortides per diferents finestres, tenim un gran ventall de possibilitats. Per exemple, la instrucció `pdf` ens permet emmagatzemar en format pdf.

Nosaltres treballarem amb fitxers de taules i els manipulem com objectes de R de classe `data.frame`. Per tant anem a centrar-nos en les funcions que fan referència a aquests objectes. Destaquem les funcions: `read.table` i `write.table`. Aquestes funcions són casos particulars de les funcions `read` i `write`, que no comentarem ja que són molt genèriques. Us animem a explorar-les.

Obriu un fitxer de text en blanc. Introduïu en columna els valors: 2.1, 3.5, 3.6, 4.9, 2.1, 5.7, 6.3, 4.7 i encapçaleu-la columna amb el nom de variable: `notes`. Guardeu el fitxer amb el nom: "notes.txt" al vostre directori de treball.

Obriu la consola de R i establiu el directori de treball amb la comanda:

```
>setwd("c:/practiques/")
```

Per a veure (o comprovar) el directori que tenim assignat fem:

```
>getwd()
```

Anem a carregar les dades del fitxer "notes.txt".

```
>read.table("notes.txt", dec=".",header=T)
```

Ara les esteu visualitzant per pantalla de la consola. Podríem guardar-les en una variable (i ens desvinculem del fitxer) fent:

```
>TaulaNotes<-read.table("notes.txt", dec=".",header=T)
>class(TaulaNotes)
>TaulaNotes
```

Això últim no és recomanable si les dades són molt grans!! Anem a crear una variable que sigui la llargada de les notes:

```
>mida<-length(TaulaNotes)
```

Fixeu-vos que no és 8 (!). La llargada no és el nombre de casos? El que succeeix és que estem demanant la llargada de la taula `TaulaNotes` i la llargada d'una taula és el nombre de variables. Si volem saber el nombre de casos de la variable `nota` que es troba a la taula `TaulaNotes`, fem:

```
>n<-length(TaulaNotes$notes)
```

Demaneu també la mitjana:

```
>mitjana<-mean(TaulaNotes$notes)
```

Ara mateix, a la sessió de R hi tenim quatre variables. Podeu comprovar-ho fent:

```
>ls()
```

Podem esborrar-ne una, per exemple:

```
>rm(mida)
```

Anem a modificar la taula. Afegirem una variable, que anomenarem `notaR`, que contingui la nota sense decimals (arrodonida).

```
>TaulaNotes<-data.frame(TaulaNotes,notaR=round(TaulaNotes$notes,0))
>TaulaNotes
```

Hi ha moltes maneres d'accedir a la informació de la taula. Per exemple, per a veure la nota arrodonida del quart cas, podem fer-ho de dues maneres:

```
>TaulaNotes[4,1]
>TaulaNotes$notaR[4]
```

Per acabar guardarem en un nou fitxer la taula que hem creat:

```
>write.table(TaulaNotes, "notes_finals.txt")
```

Podeu obrir-lo i comprovar que tot hagi anat bé.

Ara anem a fer un gràfic:

```
>hist(TaulaNotes$notes)
```

Així la sortida del gràfic és a la consola o en una nova finestra. Si volem que la sortida (és a dir, el gràfic, el dibuixet) estigui en un fitxer amb format “pdf”, fem:

```
>pdf("grafic_notes.pdf")
>hist(TaulaNotes$notes)
>dev.off()
```

Ara no ens el visualitza, ja que la sortida ha estat en un nou *device* “pdf”. Amb el `dev.off()` hem tancat aquest *device*, i ja podeu comprovar que el nou fitxer és a la vostra carpeta. El mateix es pot fer amb altres formats, cal que mireu en quins formats pot exportar la vostra versió de R, fent per exemple:

```
>help(Devices)
```

### 3.2.2 Format “.Rhistory”: Històric

Segurament ja haureu observat que podeu accedir a les instruccions que heu anat introduint a la consola movent les fletxes del teclat. Podríem dir que s'estan emmagatzemant en una “memòria d'instruccions”. Podem guardar l'històric, és a dir, les instruccions que hem anat introduint, fent:

```
savehistory("(nom_fitxer).Rhistory")
```

Això no és més que un fitxer ASCII que podem visualitzar i modificar amb un editor de text. Per exemple, podem fer copiar-enganxar les comandes directament del fitxer i executar-les a la consola.

L'avantatge d'utilitzar l'extensió “Rhistory” és que podem afegir les instruccions a la ‘memòria de les instruccions’. Això ho fem amb la instrucció:

```
loadhistory("(nom_fitxer).Rhistory")
```

Anem a guardar l'històric de la sessió

```
>savehistory("historial_pr.Rhistory")
```

Ara tanquem el R i tornem a obrir una sessió nova. Carreguem l'històric:

```
>loadhistory("historial_pr.Rhistory")
```

Fixeu-vos que ara teniu instruccions a la “memòria d'instruccions”. Obriu el fitxer “historial\_pr.Rhistory” amb un editor bàsic. Copieu i enganxeu el contingut a la consola. Ja estem com abans de tancar la sessió!

### 3.2.3 Format “.Rdata”: Àrea de treball

Podem guardar l'àrea de treball, és a dir, les variables i funcions addicionals que tingui la nostra sessió, fent

```
save.image("(nom_fitxer).Rdata")
```

I carregar-ho fent

```
load("(nom_fitxer).Rdata")
```

Aquest fitxer és binari, per tant, no proveu de visualitzar-lo. La instrucció “save.image” és un cas particular de la instrucció “save”, que és molt més complexa. Us animem a explorar-la.

Fixeu-vos que en cas de tenir una taula de dades en memòria, aquesta instrucció l'emmagatzema juntament amb d'altres coses, i per tant no hi podreu accedir.

Anem a guardar l'àrea de treball:

```
>save.image("memo.Rdata")
```

Esborreu un objecte dels que teniu en memòria, per exemple:

```
>rm(mitjana)
```

Feu `ls()` per a comprovar-ho i carreguem

```
>load("memo.Rdata")
```

Feu `ls()` per a veure que hem recuperat la variable esborrada!

### 3.2.4 Format “.R”: Scripts

Podem crear les nostres pròpies funcions, emmagatzemar-les en format ASCII, i carregar-les en memòria per a poder usar-les durant una sessió. Per això només cal obrir un editor bàsic, (o de scripts), escriure la funció i guardar-la amb l'extensió “.R”. Per carregar-la en la memòria de la sessió, utilitzarem la instrucció:

```
source("(nom_fitxer).R")
```

El fitxer pot contenir, una o més funcions o comandes. Recordeu que tot ha d'estar al mateix directori de treball i que el símbol # permet escriure comentaris que no seran interpretats com a comandes.

Anem a crear una funció:

```
>fsuma<-function(a,b)
{
  fsuma<-a+b
}
```

Proveu d'utilitzar-la,

```
>sol<-fsuma(2,3)
```

Quines variables tenim en memòria ara?

```
>ls()
```

Obriu un editor de text, o bé un script nou. Anem a copiar-hi la funció que hem creat per consola. Guardem aquest fitxer per exemple com ‘FuncionsMeves.R’. Ara, obrim una sessió de consola nova. Si fem:

```
>sol<-fsuma(2,3)
```

Obtenim un error. Carreguem el nostre fitxer de funcions:

```
>source("FuncionsMeves.R")
```

Torneu a provar

```
>sol<-fsuma(2,3)
>sol
```

### 3.3 Construccions i conceptes generals

En general, els conjunts de dades s'organitzen en la forma d'objectes de la classe `data.frame`. Aquests objectes s'assemblen en certa manera a les matrius. Tanmateix, en R les matrius són objectes d'una classe diferent, la classe `matrix`. La instrucció `data.frame` permet convertir vectors o llistes i matrius en objectes de classe `data.frame`.

Les instruccions `seq` i `rep` serveixen per a generar vectors o llistes a partir de seqüències i repeticions:

```
>seq(1,3)
>rep(1,5)
>rep(seq(1,3),5)
>rep(c(1,"t"),5)
```

També hi ha moltes instruccions per a crear matrius apilant vectors en files (o columnes). Tot seguit veurem `rbind`, que genera una matriu apilant files, `cbind` que ho fa combinant columnes i `matrix` que les crea utilitzant el contingut i les dimensions. També ens pot servir la funció `t`, que ens permet transposar les matrius. Vegem aquestes comandes:

```
>l1<-seq(1,5);l2<-seq(2,6);l3<-seq(3,7)
>cbind(l1,l2,l3)
>rbind(l1,l2,l3)
>matrix(c(1,2,3,4),2,2)
>mat<-matrix(c(l1,l2,l3),5,3)
>mat
>t(mat)
```

Resumint, quan volem fer un estudi d'estadística descriptiva amb un cert conjunt de dades, convertim els objectes a `data.frame`, posem el nom de les variables encapçalant les columnes i si ho considerem oportú, etiquetem les files o casos amb un identificador adequat.

```
>dataset1<-data.frame(mat)
>dataset1
>names(dataset1)
>names(dataset1)<-c("l1","l2","l3")
>dataset1
>dataset2<-data.frame(l1=seq(1,3),l2=c(2))
>dataset2
>row.names(dataset2)
>row.names(dataset2)<-c("a","b","c")
>dataset2
```

En tot estudi d'estadística descriptiva és important fer una exploració dels valors NA de la taula, que poden aparèixer en alguns llocs per a denotar dades perdudes o inexistents. Podem usar la funció lògica `is.na` per a crear variables sense els valors perduts. I podem explorar-los fent `summary`:

```
>notes<-c(2.3,4.5,6.2,8.7,NA,5.4,3.2,9.1,NA,5.3,2.5)
>summary(notes)
>notes_senseNA<-notes[!is.na(notes)]
>notes_senseNA
```

També hi ha un altre valor especial, que es pot produir com a resultat d'un càlcul: el valor **NaN**. Per exemple, donen com a resultat **NaN** les operacions '0/0' i 'Inf - Inf'. En canvi no ho fan les operacions '1/0', '-1/0', 'Inf-5', '2\*Inf', '(-1)\*Inf'. Comproveu-ho. **Inf** representa el valor que habitualment anomenem *infinit*. La funció **is.na** que hem vist abans retorna **TRUE** si l'argument és un **NA** o un **NaN**. Per a distingir-los hi ha la funció **is.nan**, que només retorna **TRUE** si l'argument és un **NaN**.

Sovint és incòmode haver de treballar tota l'estona amb expressions com ara **dataset\$nota**, etc. Podem utilitzar la comanda **attach** per tal que la consola reconegui les variables del conjunt de dades i **detach** per a desfer-ho. Observeu el següent exemple:

```
>TE<-data.frame(pes=c(60,72,57,90,95,72),
+altura=c(1.75,1.80,1.65,1.90,1.74,1.91))
>ls()
>TE$pes
>pes
>attach(TE)
>ls()
>pes
>detach()
>pes
```

La comanda **ls()** no les detecta com a noves variables, però fixeu-vos que ara podeu accedir directament a les variables de la taula. Cal vigilar quan usem aquestes comandes, si les variables ja existeixen a la memòria.

## Part 4

# Exercicis

**Exercici 4.1.** Utilitzant **R**, trobeu els resultats de les següents operacions amb 6 decimals:

1.  $\ln(5) + 3^5 - \sqrt{3\pi} \sin\left(\frac{2\pi}{3}\right) - \sqrt[3]{e}$

2.  $\frac{\sqrt{3+5\pi}}{7-\sqrt[5]{2}}$

3.  $2^{-\frac{7}{13}}\left(\frac{11}{9}\right)^{-\frac{8}{7}}$

4.  $\sum_{i=763}^{825} \frac{1}{i}$

5.  $\sum_{i=4}^9 \frac{3^i i!}{i^i}$

**Exercici 4.2.** Completeu els llocs buits a la comanda `rep(seq(_,_,_),_)` de **R**, per tal que generi les següents successions (primer una i després l'altra): 20, 15, 15, 10, 10, 10, 5, 5, 5, 5  
i 20, 20, 20, 20, 15, 15, 15, 10, 10, 5.

**Exercici 4.3.** Utilitzeu les comandes de **R**

```
> choose(n,k) # combinacions de n, de k en k.  
> factorial(n) # permutacions de n.
```

per a reproduir càlculs d'alguns dels 5 primers exercicis de la llista 2 de problemes.

**Exercici 4.4.** Executeu les següents comandes de **R** i comenteu els resultats:

```
> combn(4,2)  
> combn(letters[1:5],3)
```

i també:

```
> for (n in 0:9) print(choose(n,0:n)) # triangle de Tartaglia.
```

**Exercici 4.5.** Observeu les comandes de la sessió d'exemple que trobareu a l'apèndix A del manual `R-intro-1.1.0-espanol.1.pdf`. Executeu-ne algunes i observeu els resultats.

**Exercici 4.6.** Amb una comanda com la següent podem llegir dades des d'un lloc web:

```
> read.table('http://mat.uab.cat/~sintes/dades/DATA.txt',header = TRUE)
```

Utilitzeu-la per a importar la matriu de dades d'exemple contingudes al document de text "DATA.txt", a un `data.frame` anomenat `classe`.

**Exercici 4.7.** Al **RStudio** instal·leu l'**RCommander** des del repositori CRAN, clicant el boto **Install** a la pestanya **Packages** i escrivint `Rcmdr`. Quan estigui instal·lat carregueu-lo amb la comanda `> library(Rcmdr)` o bé seleccionant el quadrat ☐ corresponent, al llistat de la pestanya dels **Packages**. Experimenteu una mica amb els menús. Exploreu el help del menu.

**Exercici 4.8.** Importeu les dades del `data.frame` classe, que heu creat abans, a l'*RCommander*. Experimenteu una mica amb els menús, per exemple:

1. **Convertir variables en factors:** Podem designar quines variables són numèriques i quines són categòriques:

→ Datos → Modificar variables del conjunto de datos activo →  
→ Convertir variable numérica en factor..

2. **Resums estadístics de les variables:** Estadísticos → Conjunto de datos activo

Surten els resums de totes les variables. En el cas de les variables numèriques trobem el valor màxim, mínim, mitjana, mediana i els quartils. Per les variables no numèriques surt el recompte de freqüències.